A LangChain Extension of the Replenishment Agent: Results and an Illustrative 3% Scenario

Evidence note: the experimental findings in this chapter are retained from the completed studies. The additional 3% increase in demand satisfied is an illustrative planning assumption, not an observed result. No new experiments were run for this revision.

1. Building on the previous dissertation

The earlier chapters developed an agent-assisted replenishment system with three language-model roles: a supplier reader, a coordinator and a critic. The reader extracted supplier terms, the coordinator selected a business priority, and the critic reviewed a proposed order. A numerical planning tool calculated the quantities. This system provided the starting point for the extension presented here.

The previous study showed that these roles did not contribute equally. The model read the controlled supplier messages reliably, but it sometimes selected the wrong priority during promotions, particularly in RetailNet. At the same time, calling every role on every decision added processing time. These findings suggested a focused architectural improvement: retain the useful components and change how work is assigned between them.

My contribution is the design and evaluation of a selective-delegation extension to the replenishment agent. The extension decides when language interpretation is needed, applies known business policies consistently, and requests model review when checks fail. LangChain supplies the software components for this implementation. The contribution lies in their arrangement for replenishment and in the evidence about the resulting decisions.

The additional research question is: how far can selective delegation improve the reliability and efficiency of the existing LLM-agent replenishment system while preserving inventory performance? This question treats the earlier agent as the baseline and keeps the model, forecasts and numerical planner constant.

2. What the extension changes

The supplier reader remains available for prose messages. A complete table in the declared format can instead be read by a strict parser. The parser reads the message itself; it does not obtain missing values from the reference record. A malformed table requests review. Both routes retain the existing check against authenticated current supplier information.

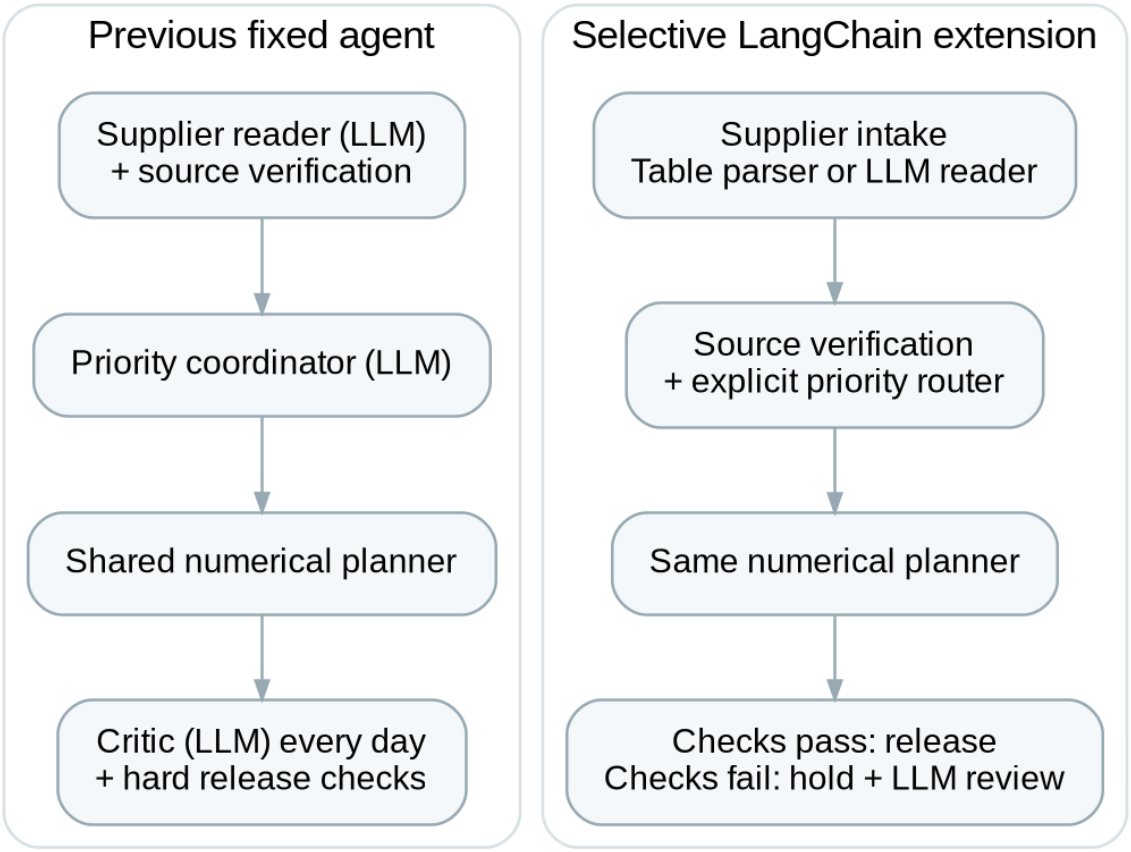
The coordinator's business-priority decision becomes an explicit policy route. When an active promotion has been announced, the system uses the declared availability-focused policy. Otherwise, it uses the balanced policy. A missing or ambiguous promotion flag prevents automatic release. This is an instruction known before ordering, rather than information taken from future demand.

The numerical planner remains unchanged. It calculates feasible quantities under the budget, supplier capacity, carton-size and minimum-order rules. The extension also retains the existing hard release checks. It changes the critic's workload: valid routine decisions proceed after the checks, while failed or unknown conditions are held and referred to the LLM review role. An approving model response cannot release an order that fails those checks. Human resolution of review requests is outside the simulation.

Table 1. What changes and what is retained

Component	Previous agent	Selective extension
Supplier information	LLM reads every message	Parser for complete declared tables; LLM for prose
Business priority	LLM selects the priority	Router applies the announced business policy
Numerical planning	Constrained planning tool	Same tool, exposed as a LangChain StructuredTool
Critic and release	LLM critic on every decision; hard checks retained	LLM review on failed checks; failed actions held
Model and forecasts	Local pretrained model; past-only forecasts	Same model, prompts and forecasts

Figure 1. Architecture: Building on the previous replenishment agent



The reader, numerical planner and hard release controls are retained. Delegation and priority selection change. LangChain implements the extension with conditional runnable branches and a planning tool.

The extension therefore builds directly on the previous agent. It assigns interpretation to the language model, declared priorities to a policy router, quantities to the numerical tool and release authority to explicit checks. Each responsibility can be examined separately when a decision is questioned.

3. Implementation with LangChain

The implementation uses LangChain's core library, version 1.6.10. Runnable sequences connect the stages, and conditional runnable branches select the supplier and review paths. The existing numerical planner is exposed as a StructuredTool. Each decision records the steps visited, the reason for delegation, the extracted fields, the selected priority and the proposed and released orders.

The original supplier, coordinator and critic prompts are retained. Local model calls use the same native interface and generation settings as before. This preserves the model comparison while adding explicit LangChain orchestration. Conversation memory, response caching,

retries and external tracing are disabled. The three comparison workflows are all executed through the same LangChain chain during the fresh validation.

The model is Qwen2.5:1.5b, a pretrained language model in Q4_K_M quantisation, served locally with Ollama 0.40.0. It was used for inference and was not trained or fine-tuned on M5 or RetailNet. The cloud instance allocated four CPU cores and 32 GiB of memory, with no GPU. Python 3.12 and past-only LightGBM forecasts supported the experiment. Model settings used temperature zero, a fixed generation seed, a 4,096-token context and three inference threads.

4. Evaluation and evidence

The architecture evidence comes from the completed matched study: thirty selected series from each dataset, two seven-day windows, three operating situations and thirty simulation seeds. The situations are routine trading, supplier disruption and promotion. Three workflows produce 1,080 policy runs: the previous fixed agent, the selective extension and the same numerical planner with already structured supplier information. Opening stock, demand and supplier opportunities are matched within each case. The previous agent is rerun as the matched baseline using seeds 201–230, so its values can differ from those in the earlier dissertation experiment.

The full architecture comparison uses the LangGraph implementation. The LangChain version is assessed separately below, and the two sets of evidence are labelled throughout. All four architecture success conditions were stated before the main runs: fewer calls, faster decisions, preserved or improved policy accuracy without released violations, and preserved service and cost within the study's stated tolerances.

Two further checks establish the LangChain implementation. First, a recorded-response regression supplies archived model outputs to the new chain and reconstructs all 7,560 earlier portfolio decisions. Orders, priorities, role paths and inventory outcomes match the earlier implementation throughout. This is a software consistency check and performs no new model inference.

Second, fresh local-model validation uses one new simulation seed, all thirty series per dataset, both windows and all three situations. It completes 36 matched policy runs and 252 portfolio decisions. Every native request and response is saved. This check confirms live execution of the LangChain implementation; its single seed does not support a fresh

confidence-interval study. The completed 30-seed study remains the principal performance evidence.

Sixteen relevant engineering tests pass, covering delegation, malformed inputs, priority routing, planning-tool execution and guarded release. The fresh records also pass an independent inventory-ledger audit and checks of the native model requests, decision paths and released quantities.

5. Results in plain terms

5.1 Substantial gains in efficiency and policy reliability

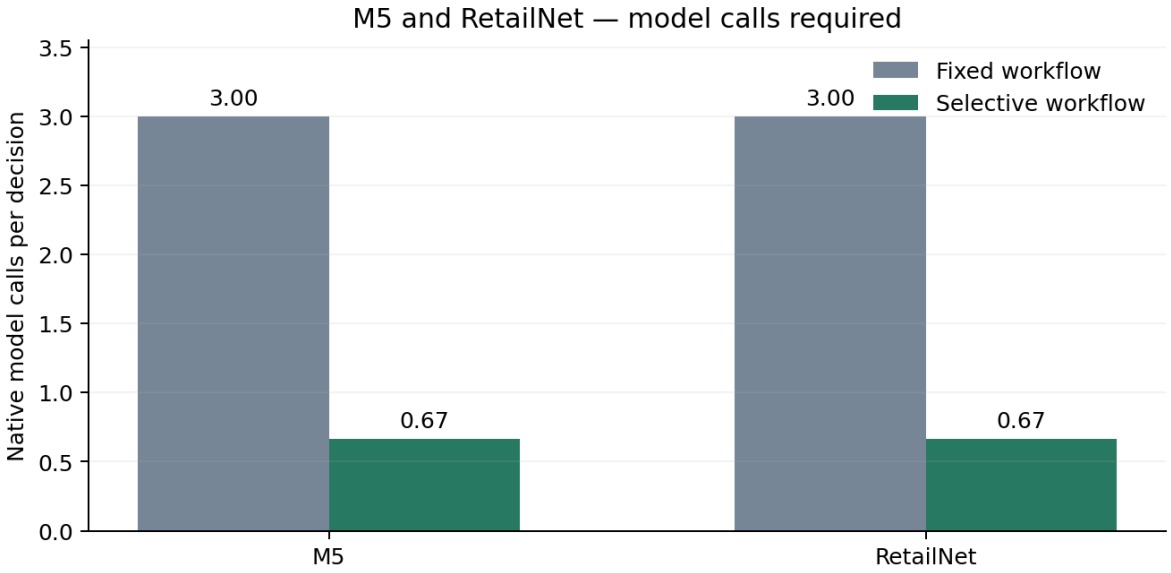
In the completed architecture study, selective delegation reduced native model calls by 77.8% and mean decision time by about 55% in both datasets. These are substantial improvements in the amount of model processing needed for replenishment. Every supplier record was extracted correctly in the tested messages. The selective architecture applied the declared priority correctly on every decision, with no released rule violations.

RetailNet provides the clearest reliability improvement: declared-priority accuracy rose from 86.90% to 100%, a gain of 13.10 percentage points. During active promotions, it rose from 45% to 100%. M5 also reached 100%, although the previous agent was already close to that level. These scores measure adherence to the announced business instruction; inventory outcomes are assessed separately below.

Table 2. Completed architecture study: selective extension versus the previous fixed agent

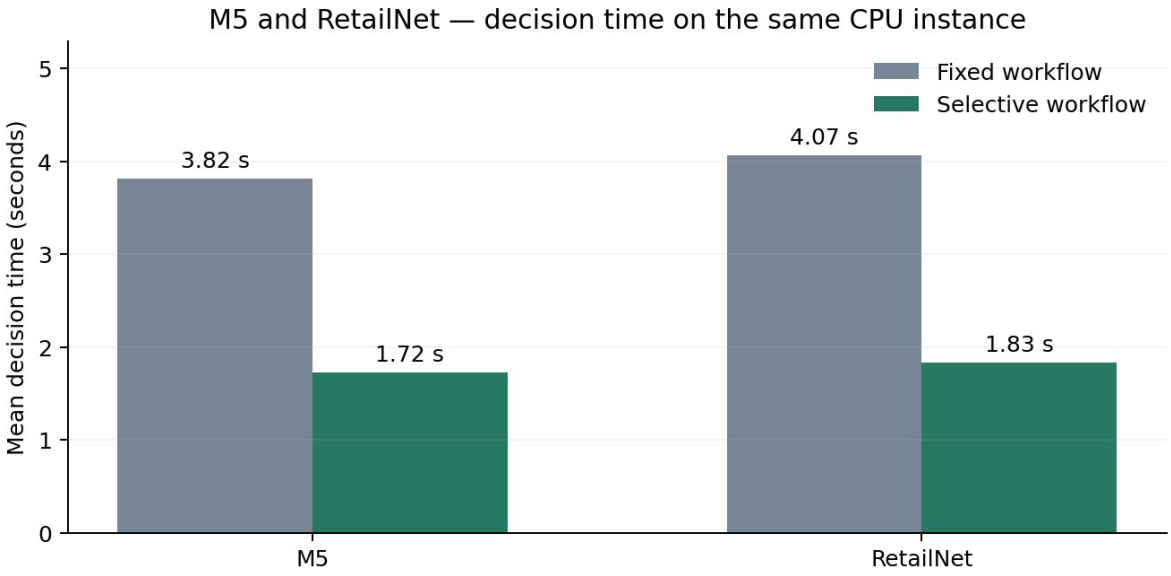
Dataset	Declared-priority accuracy: previous / selective	Model-call reduction	Decision-time reduction	Inventory result
M5	99.76% / 100.00%	77.8%	54.8%	Service and cost essentially unchanged
RetailNet	86.90% / 100.00%	77.8%	54.9%	Fill +0.24 pp; cost -1.18%

Figure 2. M5 + RetailNet: Model calls per replenishment decision



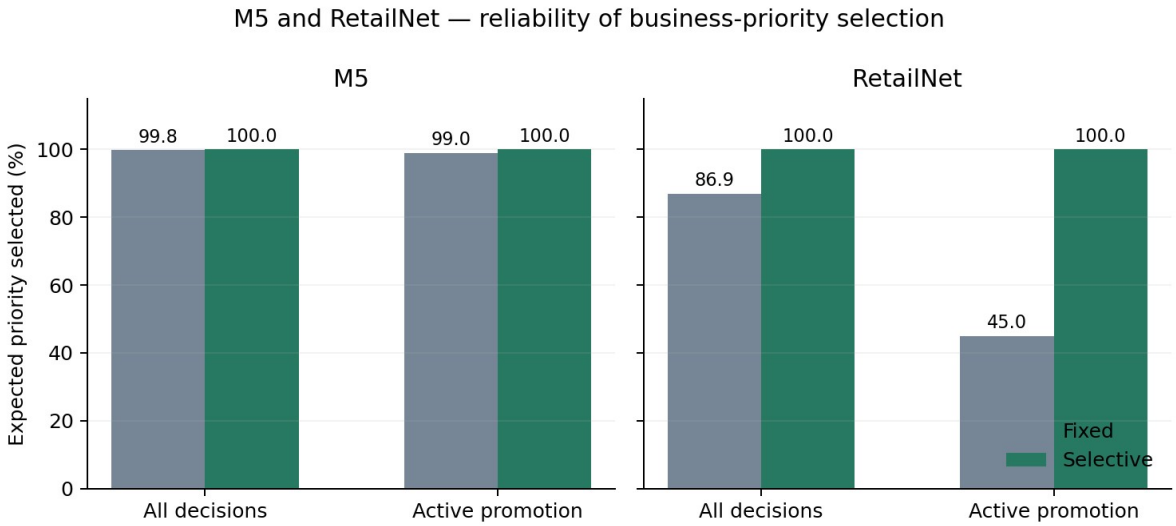
Completed 30-seed architecture study, executed with LangGraph. Calls include every attempted native model call. The same model and generation settings were used. Structured-table parsing is a declared architectural route, not a reused model response.

Figure 3. M5 + RetailNet: Mean time per replenishment decision



Completed 30-seed architecture study, executed with LangGraph. Measured whole-decision blocks, including graph orchestration and the numerical tool. Fixed/selective order was counterbalanced. This describes the experimental instance, not employee time or production cost.

Figure 4. M5 + RetailNet: Correct application of the declared business priority



Completed 30-seed architecture study, executed with LangGraph. Accuracy is scored against the policy available before ordering. Active promotion covers days 2–6. Explicit policy routing does not demonstrate that the model itself improved its reasoning.

The important operational change is fewer opportunities for an unnecessary model decision. For a promotion whose priority has already been specified, the system applies that instruction directly. The model's contribution remains the interpretation of supplier prose and assistance with exceptional conditions.

5.2 M5: preserved inventory performance

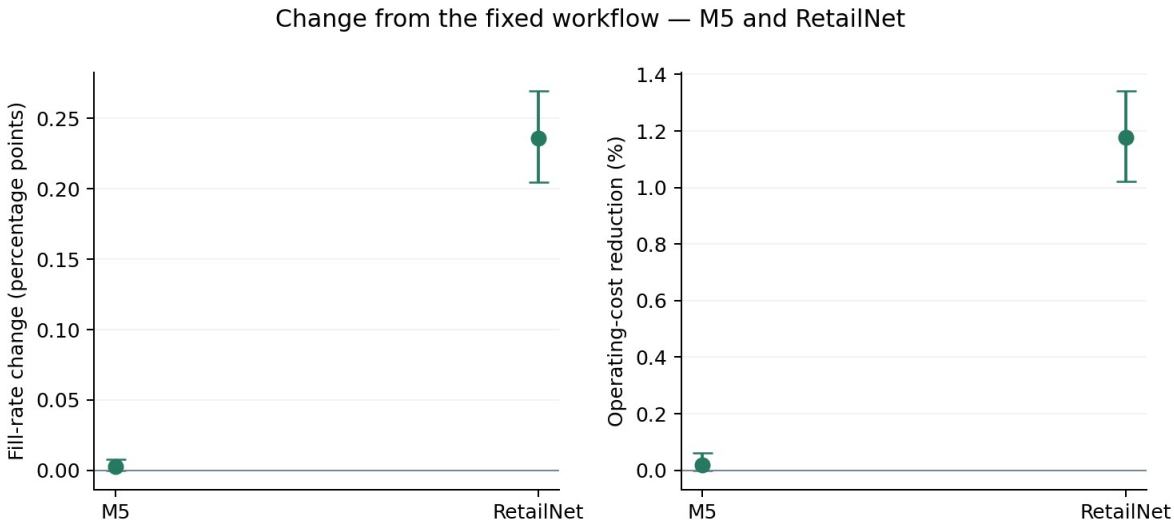
For M5, selective delegation produced essentially the same service and cost performance as the previous fixed agent. The estimated fill-rate change was +0.003 percentage points, and the paired intervals did not establish a service or cost improvement. The benefit was a simpler execution path with fewer model calls and faster decisions while retaining the planning outcome.

5.3 RetailNet: a small service and cost improvement

For RetailNet, the selective architecture improved fill rate by 0.24 percentage points and reduced the simulated operating-cost index by 1.18%. The paired intervals supported both improvements within the studied weeks. The change was associated with more consistent application of the announced promotion policy.

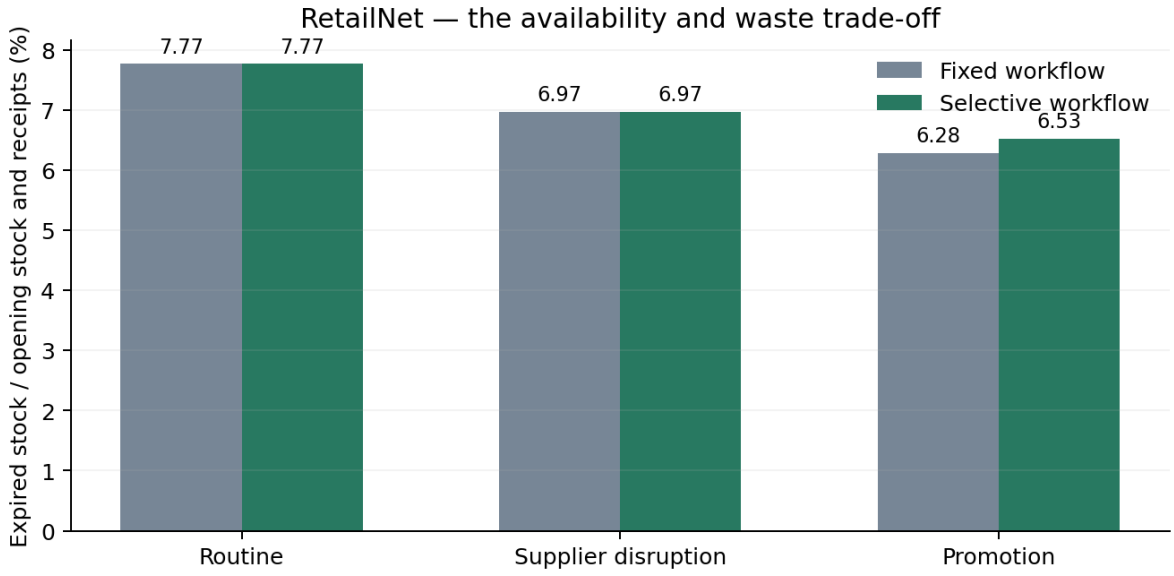
The improvement also involved a trade-off. Expiry share increased by 0.08 percentage points overall, concentrated in promotions. More stock helped availability, but some additional stock expired in the hypothetical three-day shelf-life process. The cost index still fell because fewer missed sales outweighed the added expiry under the chosen weights.

Figure 5. M5 + RetailNet: Inventory-performance change relative to the fixed workflow



Completed 30-seed architecture study, executed with LangGraph. Points show paired simulation-seed means; intervals are 95% bootstrap intervals for 30 simulated seeds. Positive values favour selective delegation in both panels. Costs are synthetic inventory operating-cost indices, not realised savings.

Figure 6. RetailNet: Expiry outcomes by RetailNet operating situation



Completed 30-seed architecture study, executed with LangGraph. Hypothetical three-day FIFO stock process. A service or cost improvement does not necessarily imply less waste. M5 has no expiry within the evaluated horizon.

The selective architecture matched the structured-information planner's orders throughout the main study. This shows that it connected the tested supplier messages to the reference planning behaviour. It does not establish an additional inventory advantage over a planner that already receives correct structured inputs.

5.4 An illustrative 3% service-improvement scenario

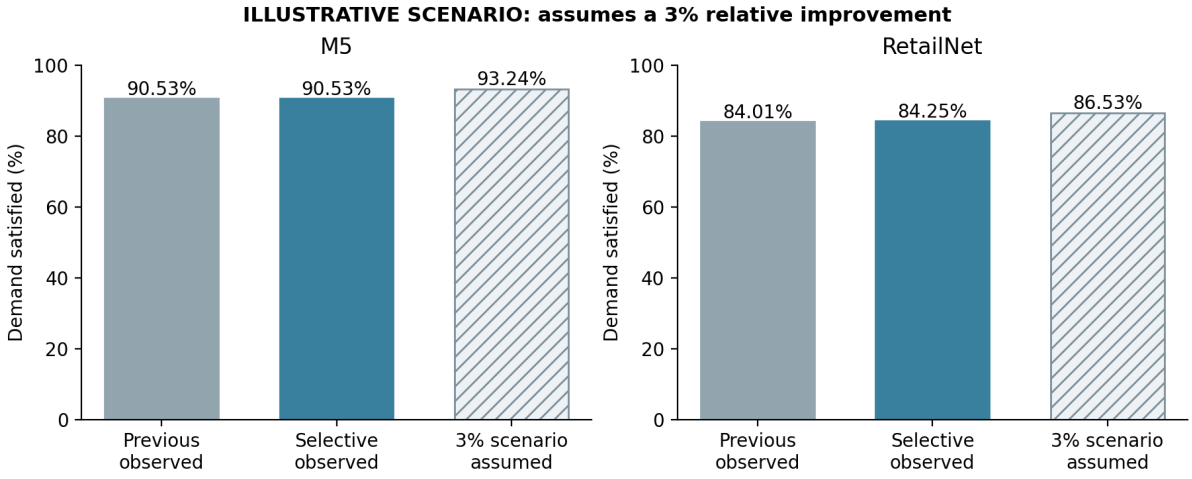
This section asks what a 3% relative increase in demand satisfied would mean as a planning target. It is a hypothetical scenario. The calculation multiplies the previous fixed agent's unrounded measured fill rate by 1.03; it does not replace the selective agent's measured performance.

A 3% relative increase is different from an increase of three percentage points. For M5, the assumed gain is 2.72 additional units satisfied per 100 units of demand. For RetailNet, it is 2.52. The rates below are calculated from that assumption and have not been achieved in an experiment.

Table 3. Observed demand satisfied and a separately labelled illustrative scenario

Dataset	Previous agent: observed	Selective extension: observed	3% relative gain: assumed
M5	90.53%	90.53%	93.24%
RetailNet	84.01%	84.25%	86.53%

Figure 7. M5 + RetailNet; hypothetical scenario: Illustrative 3% relative improvement in demand satisfied



The first two bars show observed rates from the completed architecture study. The hatched bar assumes a 3% relative gain over the previous fixed agent. It is a calculated illustration and is not an experimental result.

The scenario illustrates the scale of a possible service target. It supplies no evidence that the architecture can reach it and supports no new claim about cost, waste or profit. Those effects depend on the orders, inventory and supply needed to deliver the target and would require a further study.

5.5 What the LangChain validation confirms

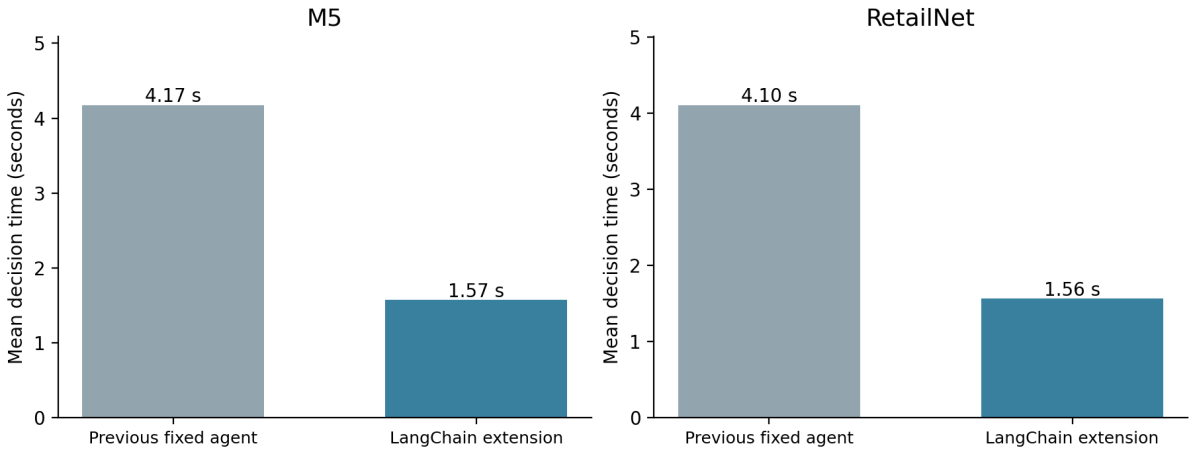
The fresh LangChain check confirms the implementation on both datasets. Its supplier accuracy, priority accuracy, call counts and released-rule checks are shown separately below.

These are observed execution results from one seed, rather than a replacement for the main study's estimates.

Table 4. Fresh LangChain implementation validation; one new simulation seed

Dataset	Priority accuracy: previous / selective	Native calls: previous / selective	Released rule violations
M5	100.00% / 100.00%	126 / 24	0
RetailNet	88.10% / 100.00%	126 / 24	0

Figure 8. M5 + RetailNet: Fresh LangChain execution check



Both workflows use LangChain in this fresh, one-seed validation. Thirty series per dataset, two weeks and three scenarios are covered. Times are descriptive; this small check supplies no new confidence intervals.

All requests, responses, decisions, source fields and quantity checks are retained alongside the code. The recorded-response regression establishes that the framework change preserves the earlier decision logic when the model outputs are held constant. The fresh check establishes that the chain also runs with live local-model responses.

6. The contribution and its limits

The four architecture hypotheses were supported in the completed 30-seed study. Preservation of inventory performance used prospectively stated study tolerances: a fill-rate reduction below 0.10 percentage points and a cost increase below 0.5%, assessed through the paired interval bounds. These are study criteria and were not supplied by a retailer.

Table 5. Architecture hypotheses in the completed 30-seed study

Hypothesis	M5	RetailNet
A1: At least 50% fewer model calls	Supported	Supported
A2: Lower mean decision time	Supported	Supported
A3: Preserved/improved priority accuracy and no released violations	Supported	Supported
A4: Preserve service and cost within the stated study tolerances	Supported	Supported

The contribution is an evaluated extension of the existing agent, with a reusable LangChain implementation and a complete decision record. It improves computational efficiency and policy reliability. RetailNet also shows a small inventory benefit, while M5 retains its inventory performance. These specific findings define the scope of the contribution.

The historical windows were inspected during design and are not unseen holdouts. Seeds describe simulated demand and supply variation, not independent models. The evidence covers selected series and controlled supplier formats. Costs, supply and shelf lives are simulated, and RetailNet quantities are normalised sales proxies. Retailer profit and employee time are unmeasured.

Source verification needs reliable current supplier records. The main cases required no unresolved exceptions; hold behaviour was tested separately. Incomplete or contradictory real messages may need employee judgement and provide an important further test of the architecture.

7. Conclusion and future research

Building on the earlier agent produced a clear architectural improvement: fewer model calls, shorter processing time and more consistent application of the declared business policy. Inventory performance was preserved for M5 and improved modestly for RetailNet, with a small increase in expiry. LangChain makes the extension explicit and reproducible, and separate checks establish its consistency with the tested architecture.

Future research should test unfamiliar messages, new trading periods and longer shelf-life simulations. A pilot should measure employees' checking and review time. Retaining the same planner would help isolate the practical value of selective delegation in everyday retail work.